

Notebook Construction and Audit Strategies

This section outlines two complementary frameworks for working with computational notebooks in machine learning and data science contexts: a **Construction Framework** for authoring new notebooks, and an **Audit Framework** for reviewing existing ones. Keeping these separate prevents the common mistake of evaluating while still constructing, which leads to incomplete analysis of both.

The two frameworks operate within a feedback loop, illustrated in Figure 1.

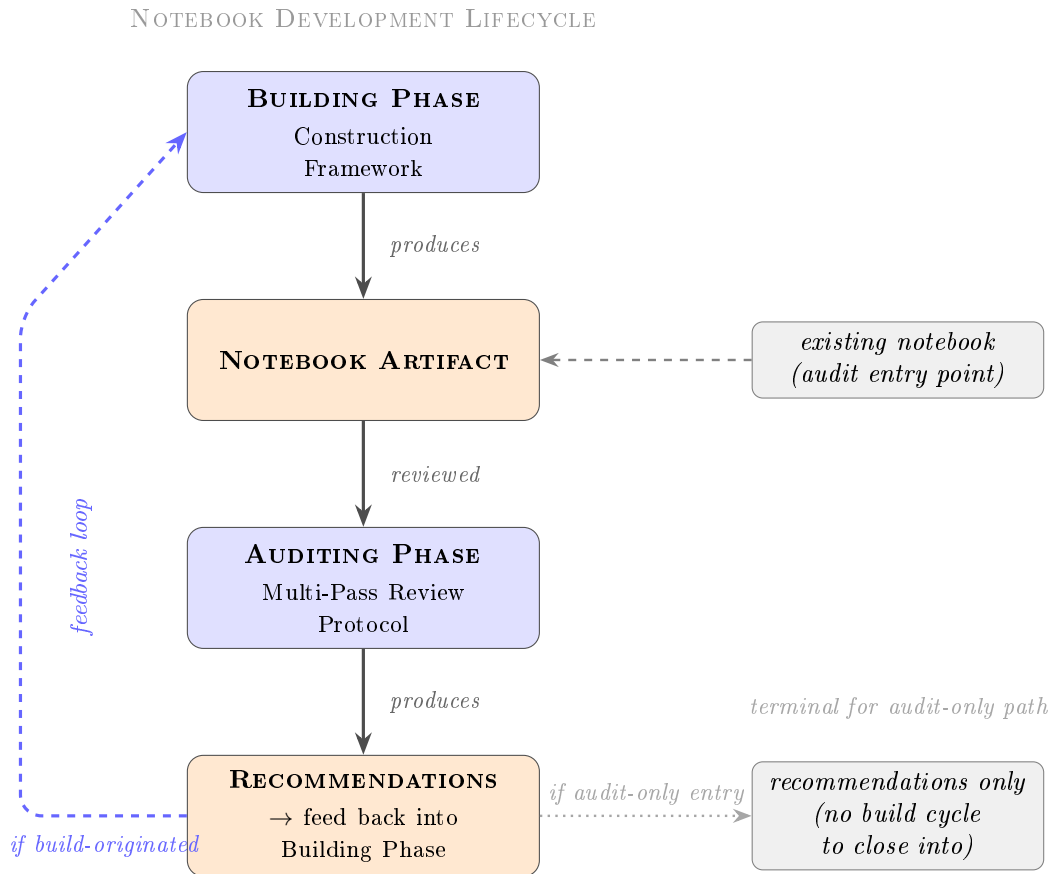


Figure 1: The Notebook Development Lifecycle. The **Construction Framework** (Part I) produces a notebook artifact, assessed by the **Audit Framework** (Part II) via a six-pass review protocol. For notebooks that originate in the Building Phase, the resulting recommendations feed back into a revised construction cycle, forming a closed feedback loop. An existing notebook (dashed entry) may instead enter the lifecycle directly at the artifact stage for audit-only workflows; in that case recommendations are a terminal deliverable rather than a loop closure, unless the notebook is subsequently routed into the Building Phase.

Part I — Notebook Construction Framework

The Construction Framework is a *forward-looking prescription*: it guides the notebook author in producing a well-structured, reproducible, and maintainable artifact from the ground up. It is organized into three sequential phases.

Phase 1 — Scaffold: Establish the structural and reproducibility foundation before writing any logic:

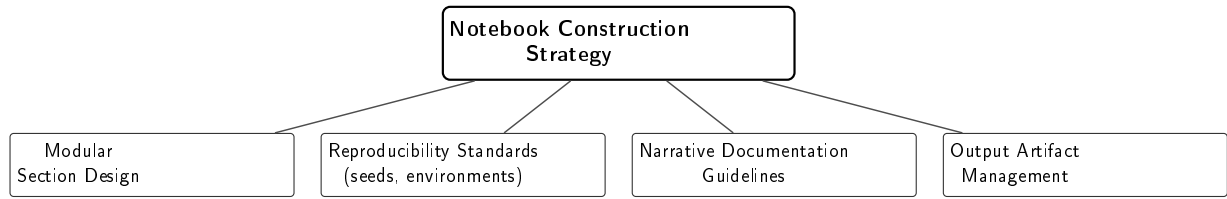


Figure 2: High-level pillars of the Notebook Construction Framework.

- Define the notebook’s **single responsibility** — one notebook should correspond to one coherent workflow or experiment.
- Create standard section headers as markdown cells, following this canonical order:
 1. Environment & Dependencies
 2. Configuration & Global Parameters
 3. Data Ingestion
 4. Preprocessing & Feature Engineering
 5. Model Definition & Training
 6. Evaluation & Metrics
 7. Artifact Export (models, plots, reports)
 8. Conclusions & Next Steps
- Pin the execution environment upfront via `requirements.txt`, `conda.yaml`, or `pyproject.toml`.
- Set global reproducibility controls: random seeds, deterministic flags, and device configuration.

Phase 2 — Write: Author each section incrementally, following these discipline rules:

- Work through **one section at a time** (Divide and Conquer principle).
- Accompany every code cell with a markdown explanation of intent and expected output.
- Avoid burying logic inside loops or helper calls without commentary.
- Prefer **explicit variable passing** over hidden or global state to maintain cell independence.
- Summarize repetitive code patterns with a general description and highlight any meaningful variations; three or more similar blocks should be treated as a refactoring candidate.
- Route all persisted outputs (models, plots, reports) through a single, versioned export convention defined at the start of the section — never write artifacts ad hoc from arbitrary cells.

Phase 3 — Validate During Writing: Continuously verify correctness as each section is completed:

- Restart the kernel and run all cells after completing each section.
- Confirm **cell idempotency**: every cell should produce the same result when re-executed independently.

- Verify that outputs match expectations before proceeding to the next section.
- Ensure cell execution order is strictly linear and free of hidden dependencies.
- Confirm that exported artifacts land in the designated output location with the expected naming and versioning convention before moving to the next section.

Part II — Notebook Audit Framework

The Audit Framework is a *backward-looking diagnostic*: it guides a reviewer, collaborator, or AI assistant in systematically assessing the quality, correctness, and risk profile of an existing notebook.

To execute this efficiently, the reviewer should employ guiding tactics such as dividing the notebook into logical modules, prioritizing critical ML pipeline blocks over repetitive exploration cells, using hierarchical descriptions (high-level summaries first, drill-downs later), and summarizing repetitive code patterns rather than analyzing each individually. When a human collaborator specifies focus areas in advance (e.g., "check only for data leakage" or "skip code style"), the reviewer should scope the six-pass review protocol to those areas rather than executing all six passes uniformly.

The framework is formalized as a six-pass review protocol, applied sequentially. Any pass included in scope is executed in full, following its own deliverable requirements below.

Where a pass deliverable is a score, the reviewer rates on a three-level scale — *Low Risk*, *Moderate Risk*, *High Risk* — based on the number and severity of unresolved items identified within that pass, rather than a numeric or percentage scale.

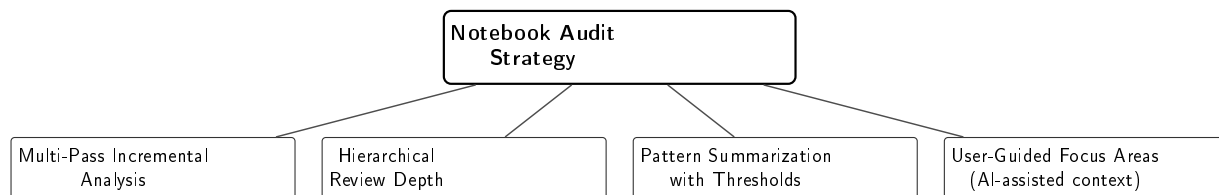


Figure 3: High-level pillars of the Notebook Audit Framework.

Pass 1 — Structural Overview: Establish a high-level map of the notebook before examining any logic:

- Record any user-specified focus areas or exclusions that will scope the remaining five passes.
- Assess whether the notebook has clear, navigable section headers.
- Confirm that the notebook has a single, coherent purpose.
- Verify that cell execution order is linear and safe.
- Identify any immediately visible red flags (e.g., missing outputs, broken imports, orphaned cells).
- *Deliverable:* Section map, preliminary red flags list, and recorded focus-area scope (if specified).

Pass 2 — Reproducibility Check: Assess whether the notebook can be reliably re-executed:

- Verify that dependencies are declared and version-pinned.
- Confirm that random seeds are set globally and consistently.

- Flag any hardcoded file paths, credentials, or environment-specific assumptions.
- Determine whether the notebook can run end-to-end on a fresh kernel without manual intervention.
- *Deliverable:* Reproducibility risk score (Low / Moderate / High).

Pass 3 — Data Integrity Review: Examine the data pipeline for correctness and leakage risks:

- Confirm that the train/test split is performed **before** any preprocessing step.
- Identify signs of **data leakage** (e.g., scaling fitted on the full dataset).
- Verify that missing data is handled consistently across splits.
- Check that data types, shapes, and distributions are validated at ingestion.
- *Deliverable:* Data pipeline integrity report.

Pass 4 — ML Correctness Audit: Evaluate the soundness of the machine learning methodology:

- Confirm that the evaluation metric is appropriate for the task and class distribution.
- Verify that cross-validation is applied correctly and without leakage.
- Ensure hyperparameter tuning is performed on validation data only, never on test data.
- Check that model performance is contextualized against a meaningful baseline.
- *Deliverable:* ML correctness checklist.

Pass 5 — Code Quality Review: Assess the maintainability and clarity of the code:

- Flag repetitive code blocks that exceed the three-block threshold as refactoring candidates.
- Identify dead code, unused imports, and redundant variable assignments.
- Assess whether variable and function names are meaningful and consistent.
- Check for bloated cell outputs (e.g., printing entire dataframes or raw tensors).
- *Deliverable:* Code quality and smell report.

Pass 6 — Deployment Readiness: Determine whether the notebook is suitable for production or publication:

- Verify that model artifacts are explicitly saved with versioned filenames.
- Confirm that inference logic is separable from training logic.
- Check that compute and memory constraints are documented or profiled.
- Ensure a complete environment export is present and up to date.
- Flag any data privacy or PII handling concerns.
- *Deliverable:* Deployment readiness score (Low / Moderate / High).

Applying the Two Frameworks

The appropriate framework depends on the context of engagement with the notebook:

Use the Construction Framework when: starting a new notebook or experiment; restructuring a disorganized notebook by discarding its existing structure and rebuilding section organization from scratch; or creating a reusable notebook template for a team or project.

Use the Audit Framework when: reviewing a collaborator's notebook; preparing a notebook for production deployment or academic publication; conducting a pre-merge or pre-deployment review; or using an AI assistant to analyze submitted notebook code.

Use Both Frameworks iteratively when: authoring and self-reviewing during active development; collaborating or pair-programming in real time; or refactoring an existing notebook into a cleaner, production-ready version while preserving its existing section structure and logic.

The Construction Framework and Audit Framework complement each other within a feedback loop: the construction phase produces a notebook artifact, the audit phase diagnoses its weaknesses, and the resulting recommendations feed back into a revised construction cycle. This closure applies to notebooks that begin in the Building Phase; for notebooks entering via the audit-only path, the loop remains open unless the notebook is subsequently taken up for construction, as illustrated in Figure 1.

Prompt Crafting

For implementing an LLM prompt engineering framework, the following prompt-draft iterations are presented in increasing order of complexity:

Lap 1. Coarse evaluation

TASK: Analyze the provided Jupyter Notebook and produce a structured audit. Proceed to systematically analyze the codebase, examining each section dependently. After all reports have been generated, compare them without looking at the code. once you have a comparison, compare the mission report to the code base and point out any discrepancies

PHASE 1 — DOCUMENTATION [Section 1: Overall Purpose — 2-3 sentences] [Section 2: Code Block Descriptions — numbered by executable cells only, with merged categories: Logic and Flow, Data and State, Operations and Methods, Side Effects] [section 3: Pseudocode any functions that seem too complicated for the naked eye. Revise all comments. Suggest a new code naming convention.] [Section 4: Pipeline Summary — 3-5 sentences if dependencies exist]

PHASE 2 — CRITICAL AUDIT [Evaluate and score the ORIGINAL CODE (not your descriptions) for: - Cross-block consistency in data types and variable naming - Redundant computation or unnecessary I/O - Contradictory logic or unreachable branches - Missing error handling or validation]

Suggest a Gate decision

add fingerprint "lap 0" and timestamp

PHASE 2 — CRITICAL AUDIT [Evaluate and score the ORIGINAL CODE (not your descriptions) for: - Cross-block consistency in data types and variable naming - Redundant computation or unnecessary I/O - Contradictory logic or unreachable branches - Missing error handling or validation]

Suggest a Gate decision

add fingerprint "lap 0" and timestamp

Lap 2. More refined evaluation

Task: Analyze the provided Jupyter Notebook code and extract the following structured information. Number code blocks sequentially (Block 1, Block 2, etc.) based on top-to-bottom visual appearance, strictly ignoring Jupyter's native execution counters (e.g., []).

1 - Overall Purpose / Goal Provide a clear and concise summary (2-3 sentences maximum) of the notebook's main objective, including:

- The problem it solves - The type of input it expects - The output or result it produces 2 - Conceptual Description of Each Code Block Apply the following rules for processing cells:

Consolidation: Group all library imports, package installations (e.g., !pip), and magic commands (e.g., Filtering: Skip and do not include pure markdown cells, raw text cells, empty cells, or cells that throw execution errors in the block list below. For all remaining valid executable code blocks, extract:

Logic Flow: Step-by-step reasoning behind the code's execution. Data Transformations: What goes in (inputs) and what comes out (outputs) regarding data shapes/types (e.g., "Transforms DataFrame X from wide to long format"). Model/Algorithm Operations: Any ML, statistical, or computational methods applied. Key Variables/Parameters: Important identifiers, hyperparameters, or configurations. Side Effects: File I/O, state changes, printed output, API calls, etc. 3 - Output Format Requirements Structure your response exactly as follows, using the exact headings provided:

1 - Overall Purpose / Goal [Your summary here]

Code Block Descriptions Block 1 - Environment Setup Logic Flow: N/A Data Transformations: N/A Model/Algorithm Operations: N/A Key Variables/Parameters: [List imported libraries/versions if specified] Side Effects: [List any installations or magic command outputs]

Block 2 - [Short Descriptive Title] Logic Flow: ... Data Transformations: ... Model/Algorithm Operations: ... Key Variables/Parameters: ... Side Effects: ... [Continue pattern for all blocks...]

4 - Pipeline Summary Explain how the extracted blocks connect to form a complete pipeline:

Trace the data flow from the first computational block to the last. Detail any branching, looping, or conditional paths across blocks. 5 - Cross-Block Code Quality Audit Audit the notebook's underlying code for structural and logical issues. Address each of the following points strictly using bullet points:

Coherence and Consistency: [Note any mismatches between data transformations and downstream usage] Redundancy and Waste: [Identify any inefficient repeated computation or redundant data transformations] Contradictory Logic: [Point out any instances where later blocks logically contradict or overwrite earlier blocks] If no issues are found in a category, write "None detected."

add fingerprint "lap 1" and timestamp

Lap 2. Structured evaluation A 3-level structure for engineering prompts is assumed that maps cleanly onto the existing six passes without requiring changes to the framework itself:

1. *Level 1 — Conceptual* (Passes 1–2): Structural Overview + Reproducibility Check. Establishes whether the notebook is coherent and re-runnable at all, before any deep inspection is worth doing.
2. *Level 2 — Methodological* (Passes 3–4): Data Integrity Review + ML Correctness Audit. Addresses whether the pipeline and modeling logic are scientifically sound — the "does this work correctly" layer.

3. *Level 3 — Implementation/* (Passes 5–6): Code Quality Review + Deployment Readiness. Addresses how the working logic is written and packaged — the most granular, code-level layer.

Level 1 – Conceptual Purpose

determine whether the notebook is coherent and reliably re-executable before investing time in deeper review.

Step 1.1 — Scope the audit:

Record any user-specified focus areas or exclusions (if a collaborator has scoped the review in advance). Note this scope explicitly — it determines which of the remaining passes/levels get full attention later.

Step 1.2 — Map the structure

Assess whether the notebook has clear, navigable section headers. Confirm the notebook has a single, coherent purpose (not multiple unrelated experiments bundled together). Verify cell execution order is linear and free of hidden dependencies.

Step 1.3 — Scan for immediate red flags

Identify visible issues without deep inspection: missing outputs, broken imports, orphaned cells.

Step 1.4 — Check reproducibility inputs

Verify dependencies are declared and version-pinned (requirements.txt, conda.yaml, pyproject.toml, or equivalent). Confirm random seeds are set globally and consistently.

Step 1.5 — Check reproducibility risks

Flag hardcoded file paths, credentials, or environment-specific assumptions. Determine whether the notebook can run end-to-end on a fresh kernel without manual intervention.

Level 1 deliverables:

Section map, Preliminary red flags list, Recorded focus-area scope (if specified), Reproducibility risk score (Low / Moderate / High)

Gate decision before moving to Level 2: if the notebook fails end-to-end execution or lacks a coherent single purpose, decide now whether to proceed to Level 2 anyway (methodological review can still surface findings) or to stop and flag Level 1 as blocking — the framework doesn't mandate either, so this is your call to make consistently across audits.

You are performing a Level 1 (Conceptual) audit of a Jupyter notebook. This notebook is moderate length (approximately 400-600 lines) — read it in full before reporting; do not sample or skim sections.

Your **ONLY** objective at this level is to determine whether the notebook is structurally coherent and reliably re-executable. Do NOT evaluate code quality, ML methodology, data leakage, or deployment readiness — those belong to later audit levels and are out of scope here.

Follow these steps in order:

STEP 1 — SCOPE - Note if I have specified any focus areas or exclusions for this audit. If none given, assume full Level 1 scope.

STEP 2 — STRUCTURE MAP - List the notebook's section headers (markdown cells) in order. - State whether the notebook has a single, coherent purpose, or if it bundles multiple unrelated workflows/experiments. - Verify

cell execution order is linear — flag any cell that appears to depend on a later cell, or any out-of-order execution counts visible in cell outputs.

STEP 3 — RED FLAGS SCAN - Identify immediately visible issues: missing/empty outputs, broken or unused imports, orphaned cells (code with no clear purpose or downstream use), cells left in a non-executed state.

STEP 4 — REPRODUCIBILITY INPUTS - Check whether dependencies are declared and version-pinned (look for a requirements.txt, environment.yml, pyproject.toml, or an explicit pip/conda install cell with pinned versions). - Check whether random seeds are set globally (e.g., numpy, torch, random, sklearn) and consistently applied before any stochastic operation.

STEP 5 — REPRODUCIBILITY RISKS - Flag hardcoded file paths, credentials, API keys, or any environment-specific assumption (e.g., a path that only exists on one machine). - Assess whether the notebook could plausibly run end-to-end on a fresh kernel without manual intervention. State your reasoning, not just a yes/no.

OUTPUT FORMAT — return exactly these four items, nothing else:

1. Section map (ordered list of headers, or "no clear section headers" if absent) 2. Preliminary red flags list (bullet points; write "none identified" if empty) 3. Recorded focus-area scope (state "full scope" if none was specified) 4. Reproducibility risk score: Low / Moderate / High, with a one-sentence justification tied to what you found in Steps 4-5

Do not add commentary, recommendations, or fixes at this stage — Level 1 is diagnostic only. Do not comment on code style, variable naming, or model correctness.

Level 2 – Methodological Purpose

determine whether the data pipeline and modeling methodology are scientifically sound, assuming Level 1 has already confirmed the notebook is structurally coherent and runnable.

Step 2.1 — Verify split ordering

Confirm that the train/test split is performed before any preprocessing step (scaling, encoding, imputation, feature engineering).

Step 2.2 — Check for pipeline-level leakage

Identify signs of data leakage introduced during preparation (e.g., a scaler, encoder, or imputer fitted on the full dataset rather than the training split alone). Leakage introduced later, through cross-validation or model selection, is addressed in Step 2.5 — don't double-flag it here.

Step 2.3 — Check missing-data handling

Verify that missing data is handled consistently across the train and test splits (same strategy applied to both, fit only on training data).

Step 2.4 — Validate data at ingestion

Check that data types, shapes, and distributions are validated when the data is first loaded (not assumed silently downstream).

Step 2.5 — Verify cross-validation integrity

Confirm cross-validation folds preserve the train/test separation established in Step 2.1. Check that no fold-selection, resampling, or preprocessing-within-CV step exposes validation or test data to the training process.

Step 2.6 — Check evaluation metric appropriateness

Confirm the evaluation metric fits the task and class distribution (e.g., accuracy on an imbalanced classification problem is a red flag).

Step 2.7 — Check hyperparameter tuning boundaries

Ensure hyperparameter tuning is performed on validation data only, never on the test set.

Step 2.8 — Check baseline comparison

Confirm model performance is contextualized against a meaningful baseline (e.g., a naive predictor, a simpler model, or a published benchmark) — not reported in isolation.

Level 2 deliverables:

Data pipeline integrity report (from Steps 2.1–2.4), ML correctness checklist (from Steps 2.5–2.8)

Gate decision before moving to Level 3: if pipeline-level leakage is found in Step 2.2, decide whether downstream ML correctness findings (Steps 2.5–2.8) are still meaningful to report, since a leaky pipeline can invalidate reported model performance regardless of how sound the CV/tuning procedure is. As with the Level 1 gate, the framework doesn't mandate stopping — this is a call to make consistently across your audits.

You are performing a Level 2 (Methodological) audit of a Jupyter notebook. This notebook is moderate length (approximately 400-600 lines) — read it in full before reporting; do not sample or skim sections.

Assume Level 1 (structural/reproducibility review) has already been completed. Your ONLY objective at this level is to determine whether the data pipeline and modeling methodology are scientifically sound. Do NOT comment on code style, variable naming, section headers, or deployment readiness — those belong to other audit levels and are out of scope here.

Follow these steps in order:

STEP 1 — SPLIT ORDERING - Locate where the train/test split occurs in the notebook. - Confirm it happens BEFORE any preprocessing step (scaling, encoding, imputation, feature engineering). If preprocessing precedes the split, flag this explicitly and identify the exact cell(s) involved.

STEP 2 — PIPELINE-LEVEL LEAKAGE - Identify any scaler, encoder, imputer, or feature-engineering step fitted on the full dataset rather than the training split alone. - Do NOT flag cross-validation or hyperparameter-tuning leakage here — that belongs to Step 5.

STEP 3 — MISSING DATA HANDLING - Verify missing data is handled with a strategy fit only on the training data, then applied identically to the test data. - Flag any case where the missing-data strategy is computed on the full dataset or differs between splits.

STEP 4 — DATA VALIDATION AT INGESTION - Check whether data types, shapes, and distributions are inspected or validated when the data is first loaded (e.g., dtype checks, null counts, shape assertions, distribution plots). - Flag if the notebook proceeds directly to modeling without any ingestion-time validation.

STEP 5 — CROSS-VALIDATION INTEGRITY - Confirm cross-validation folds preserve the train/test separation identified in Step 1. - Check whether any fold-selection, resampling, or preprocessing step inside the CV loop exposes validation or test data to training.

STEP 6 — EVALUATION METRIC APPROPRIATENESS - Identify the evaluation metric(s) used. - State whether the metric fits the task and class distribution (e.g., flag accuracy as a red flag on an imbalanced classification problem).

STEP 7 — HYPERPARAMETER TUNING BOUNDARIES - Confirm hyperparameter tuning is performed only on validation data, never on the test set. Identify the exact cell(s) where tuning occurs.

STEP 8 — BASELINE COMPARISON - Check whether model performance is compared against a meaningful baseline (naive predictor, simpler model, or published benchmark). - Flag if performance is reported in isolation with no baseline context.

OUTPUT FORMAT — return exactly these two items, nothing else:

1. Data pipeline integrity report (Steps 1-4): for each step, state PASS or FLAG, with the specific cell reference and a one-sentence explanation for any flag.
2. ML correctness checklist (Steps 5-8): for each step, state PASS or FLAG, with the specific cell reference and a one-sentence explanation for any flag.

If pipeline-level leakage is found in Step 2, note this at the top of your output as a blocking issue, since it can invalidate any performance numbers evaluated in Steps 5-8 — but still complete and report Steps 5-8 regardless.

Do not propose fixes or rewrite code at this stage — Level 2 is diagnostic only.

Level 3 — Implementation Purpose

determine whether the code is maintainable and clear, and whether the notebook is suitable for production or publication — assuming Level 1 has confirmed structural coherence and Level 2 has confirmed methodological soundness. Step 3.1 — Flag repetitive code blocks

Identify code blocks that exceed the three-block threshold (three or more similar blocks) and mark them as refactoring candidates.

Step 3.2 — Identify dead code and unused elements

Flag unused imports, dead code paths, and redundant variable assignments.

Step 3.3 — Assess naming quality

Evaluate whether variable and function names are meaningful and used consistently throughout the notebook.

Step 3.4 — Check output hygiene

Flag bloated cell outputs (e.g., printing entire dataframes, raw tensors, or large arrays) that clutter the notebook without adding interpretive value.

Step 3.5 — Verify artifact export

Confirm model artifacts (models, plots, reports) are explicitly saved with versioned filenames, not left only in memory or overwritten without versioning.

Step 3.6 — Check inference/training separability

Confirm inference logic is separable from training logic — i.e., a saved model could be loaded and used for inference without re-running the training pipeline.

Step 3.7 — Check resource documentation

Verify that compute and memory constraints are documented or profiled (e.g., expected runtime, GPU/CPU requirements, peak memory usage).

Step 3.8 — Check environment completeness

Confirm a complete environment export is present and up to date (consistent with what Level 1 checked for reproducibility, but here verified for deployment sufficiency specifically).

Step 3.9 — Flag data privacy concerns

Identify any handling of PII or sensitive data without appropriate safeguards (anonymization, access control, exclusion from version control).

Level 3 deliverables:

Code quality and smell report (from Steps 3.1–3.4), Deployment readiness score, Low / Moderate / High (from Steps 3.5–3.9)

You are performing a Level 3 (Implementation) audit of a Jupyter notebook. This notebook is moderate length (approximately 400-600 lines) — read it in full before reporting; do not sample or skim sections.

Assume Level 1 (structural/reproducibility review) and Level 2 (data pipeline/ML methodology review) have already been completed. Your ONLY objective at this level is to assess code maintainability and deployment readiness. Do NOT re-evaluate section structure, data leakage, model correctness, or evaluation metrics — those belong to other audit levels and are out of scope here.

Follow these steps in order:

STEP 1 — REPETITIVE CODE - Identify any code pattern that repeats across three or more cells or blocks (e.g., repeated plotting logic, repeated preprocessing steps applied separately to each feature). - Flag each as a refactoring candidate with the specific cell references.

STEP 2 — DEAD CODE AND UNUSED ELEMENTS - Identify unused imports. - Identify dead code paths (code that cannot be reached or has no downstream effect). - Identify redundant variable assignments (e.g., a variable reassigned without ever being read in between).

STEP 3 — NAMING QUALITY - Assess whether variable and function names are descriptive and consistent throughout (e.g., not 'df1', 'df2', 'tmp', 'x2' used interchangeably with meaningful names elsewhere). - Flag inconsistencies where the same concept is named differently in different cells.

STEP 4 — OUTPUT HYGIENE - Identify any cell output that prints an entire dataframe, raw tensor, or large array without truncation, sampling, or summarization. - Flag these as clutter that reduces notebook readability.

STEP 5 — ARTIFACT EXPORT - Confirm that models, plots, and reports are explicitly saved to disk with versioned or timestamped filenames. - Flag any artifact that is generated but never saved, or saved with a filename that would be overwritten on the next run.

STEP 6 — INFERENCE/TRAINING SEPARABILITY - Determine whether a saved model could be loaded and used for inference without re-running the training pipeline. - Flag if inference logic is entangled with training logic in the same cells with no clear separation point.

STEP 7 — RESOURCE DOCUMENTATION - Check whether the notebook documents or profiles compute/memory requirements (expected runtime, GPU/CPU needs, peak memory). - Flag if no such documentation exists, especially for computationally expensive cells (training loops, large matrix operations).

STEP 8 — ENVIRONMENT COMPLETENESS - Verify a complete, up-to-date environment export exists (e.g., requirements.txt, environment.yml) sufficient for someone else to reproduce a deployment-ready environment — not just "the code runs," but "the exact dependency versions are captured."

STEP 9 — DATA PRIVACY CONCERNS - Identify any handling of personally identifiable information (PII) or sensitive data. - Flag missing safeguards: no anonymization, no access control, data or credentials committed directly into the notebook or version control.

OUTPUT FORMAT — return exactly these two items, nothing else:

1. Code quality and smell report (Steps 1-4): for each step, list findings with specific cell references; write "none identified" if a step surfaces nothing.

2. Deployment readiness score: Low / Moderate / High (Steps 5-9), with a bulleted justification — one bullet per step, stating PASS or FLAG with a one-sentence reason for any flag.

If Step 8 findings overlap with a reproducibility check from Level 1, note it briefly as "consistent with Level 1 finding" rather than re-describing it in full — avoid duplicating that report.

Do not propose fixes or rewrite code at this stage — Level 3 is diagnostic only.
